

Tighter Bounds for h^+ MIP Planning: LM-Cut Strengthening and Cut Generation

Domenico Salvagnin¹ Matteo Zanella¹

¹DEI, University of Padova



UNIVERSITÀ
DI PADOVA

DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

Problem & Motivation

Classical Planning aims to find a sequence of actions, or *plan*, leading towards the goal state, from the initial state. In the delete-free relaxation, each action is composed by two sets of facts, the *preconditions* and *add effects*, which specify how actions affect a state. Our goal is to find the sequence of action that minimizes total cost.

Current state-of-the-art: MIP formulation divided in base model and acyclicity constraints, aiming to prevent cycles in the underlying causal relation graph:

- Vertex Elimination Constraints [FRR22]
- Landmark Constraints [HST12, BC11, DM26]

Core Question: can we improve the landmark generation to produce more and better landmarks?

Formal Setting

A planning task [FN71] is defined as:

$$\Pi := (P, A, s, t, c)$$

- P : set of boolean variables (*facts*)
- A : set of *actions* $a := (\text{pre}(a), \text{add}(a))$
- s, t : *initial* and *goal facts*
- $c : A \rightarrow \mathbb{R}_0^+$: non-negative cost function

A sequence of actions π is a *feasible solution* if each actions preconditions are respected before being applied $\text{pre}(a_{\pi_i}) \subseteq S_{[a_{\pi_1}, \dots, a_{\pi_{i-1}}]} := \{s\} \cup (\bigcup_{a \in [a_{\pi_1}, \dots, a_{\pi_{i-1}}]} \text{add}(a))$ for all $i \in \{1, |\pi|\}$ and the state reached contains the goal state $t \in S_\pi$.

Disjunctive Action Landmarks: set of actions $L \subseteq A$ such that each plan π must contain at least one action in L : delete-free planning can be solved as a minimum-cost hitting set problem over all its landmarks [BH10]. We can use landmark constraints to (indirectly) model acyclicity in the causal relation graph.

LM-Cut: LM-cut [HD09] is a state of the art admissible heuristic for classical AI planning, based on the iterative computation of landmarks: at each step of the process, a pcf with maximum $h^{\max}$ [BG01] is chosen, an s - t cut-set is identified yielding a landmark L , and the cost of actions in L is reduced by the minimum between their costs. The process is repeated until no cut-set of positive (reduced) cost can be found.

Strengthening LM-Cut

At each iteration, LM-Cut builds a justification graph (sub-graph of the causal relation graph) in which for each action only one precondition (the *precondition choice function* $D : A \rightarrow P$) is preserved; the facts are then partitioned into *goal* (S) and *pre-goal* ($P \setminus S$) section: facts that can reach through a (reduced) 0-cost path the goal fact t end up in the goal section. A landmark is then computed as the set of actions crossing this partition:

$$L = \{a \in A \mid D(a) \in (P \setminus S), \text{add}(a) \cap S \neq \emptyset\}$$

The landmark produced has no guarantee of being minimal, hence, before reducing the cost of the actions found, we can improve the landmark making it minimal: this produces smaller landmarks, and thus we are able to produce more of them.

We propose two different minimalization procedures: an efficient greedy heuristic and an expensive minimalization procedure [HST12].

- The greedy heuristic simply changes the definition of the landmark extracted:

$$L = \{a \in A \mid \text{pre}(a) \subseteq (P \setminus S), \text{add}(a) \cap S \neq \emptyset\}$$

Strengthening LM-Cut

The idea is that if we can simply change the pcf of some actions in the cut, we can remove redundant actions: this won't invalidate the cut, since changing these pcfs won't change neither the goal or pre-goal sections.

- The expensive approach removes as many actions from the cut, while making sure it is still a valid landmark

LM-Cut as cut separator

Core Question: can we find violated landmarks using LM-Cut? Executing LM-Cut will produce the same landmarks used for preprocessing, so simply re-executing it won't find violated landmarks.

Tweaking LM-Cut into a cut separator We can tweak LM-Cut so that it finds only violated landmarks: if we set the initial (reduced) cost of used actions to 0, LM-Cut will be looking for landmarks made only of unused actions, hence violated ones. *Note that we are not removing edges from the justification graph: this would invalidate the landmarks produced; we are instead "warm-starting" LM-Cut so that it will only look among violated ones.* The initial costs of actions in the LM-Cut execution, w.r.t. a solution $\tilde{x}$, are

$$c'(a) = (1 - \tilde{x}_a)c(a)$$

Note that with this cost assignment, LM-Cut can be used also on fractional solutions: due to the NP-Hardness of the exact fractional separator [DM26] there's no guarantee that it will find violated landmarks, hence we keep only violated ones and drop the others.

Computational Results

Figure 1 shows the optimality gap between h^{LMCut} and h^+ through a boxplot: the results show a significant reduction of the gap throughout all tested pcf [LF20] when enabling the expensive minimalization approach. This improvement comes at a cost however: LM-Cut with expensive minimalization is $2/3 \times$ slower. We tested the greedy minimalization approach, but improvements were minimal compared to the other approach while requiring almost no additional time.

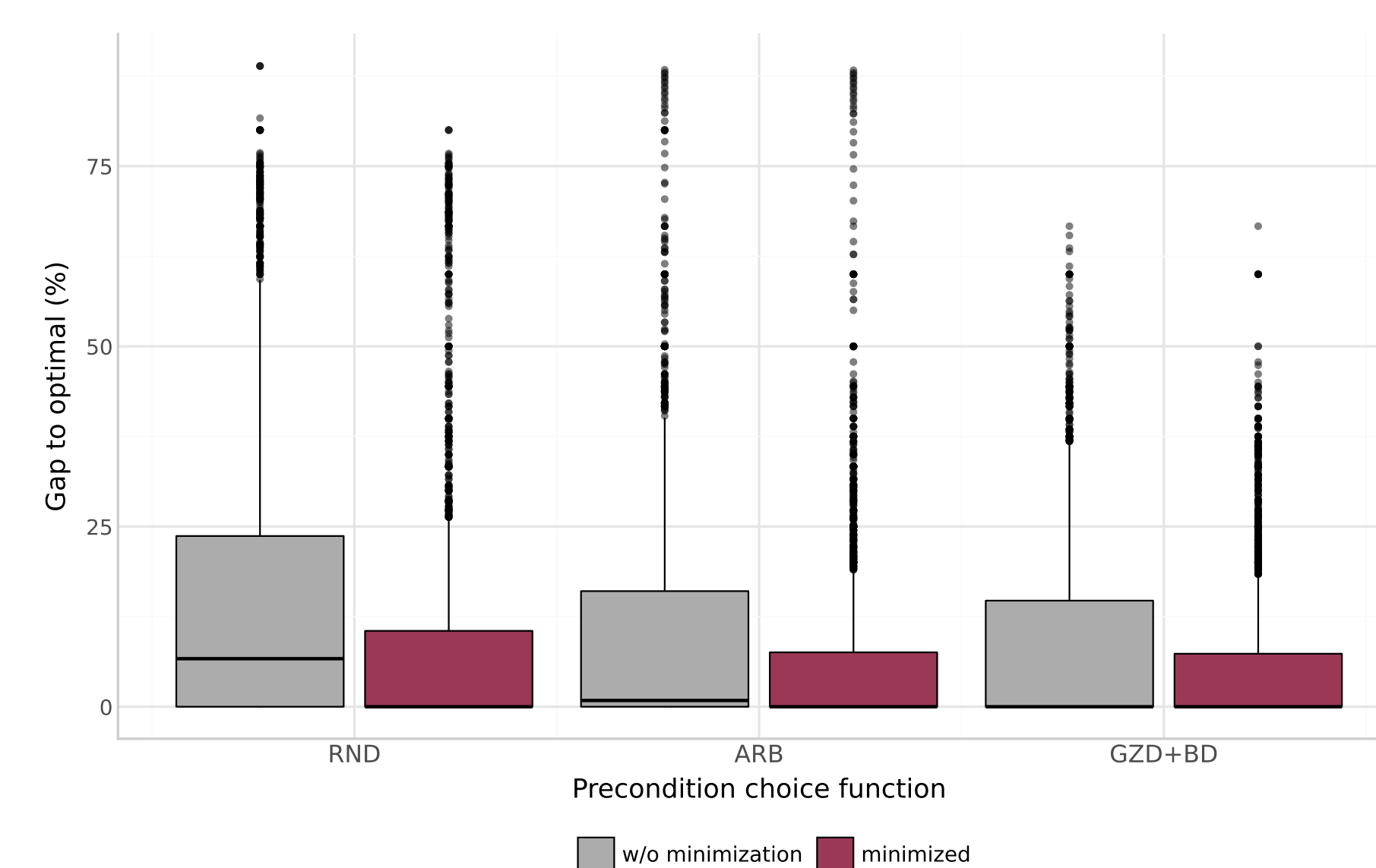


Figure: Optimality gap $(1 - h^{\text{LMCut}}/h^+)$.

The increased computational time, however, is negligible in the context of computing h^+ : we used LM-Cut to compute an initial set of landmark constraints as a warm-start to the landmark-based model and compared the different minimalization approaches (Table 1).

Category	LM-Cut			+ greedy min.			+ exact min.		
	#	Solved	Time	Solved	Nodes	Time	Solved	Nodes	Time
all	2799	2443	1.893	+21*	0.805*	0.891*	+39*	0.644*	0.863*
solvable	2539	2443	0.757	+21*	0.697*	0.818*	+39*	0.469*	0.772*
all-solvable	2373	2373	0.531	0	0.841*	0.920*	0	0.586*	0.894*

Table: Landmark model with LM-Cut warm-start (SGM time, s) and minimization variants (Δ solved instances; Nodes/Time as SGM ratios vs. baseline, < 1 is better). * statistically significant.

Computational Results

Table 1 shows a reduction in the number of nodes of 20% to 40%, a reduction of 20% to 25% in computational time and +39 solved instances: this is due to the better bound provided by minimizing LM-Cut. Table 2 instead shows the improvements that we can obtain by using first LM-Cut as cut separator for integer solutions, and then for fractional solutions as well: using LM-Cut, with exact minimalization, on integer solutions manages to further reduce by 30% the number of nodes and by 10 – 15% the total time, while finding the optimal for 47 more instances. For fractional solutions we need the more efficient minimalization procedure, hence we used the greedy one; we managed to solve 12 additional instances with a significant reduction in the number of nodes explored and gaining a few additional percentages in computational time.

		Landmarks			LM-Cut			fract LM-Cut		
Category	#	Solved	Time	Solved	Nodes	Time	Solved	Nodes	Time	
all	2799	2482	1.633	+47*	0.663*	0.890*	+59*	0.195*	0.867*	
solvable	2598	2482	0.905	+47*	0.670*	0.850*	+59*	0.200*	0.817*	
all-solvable	2422	2422	0.510	0	0.883	1.007	0	0.268*	0.998	

Table: Comparisons between different cut separator strategies.

Conclusions

- **Speed:** The proposed techniques, sum up to a total **30% reduction in computational time** and of **85%** in the **number of nodes**, with respect to the baseline.
- **Gap:** By strengthening LM-Cut we manage to produce **more and smaller landmarks**, which leads to **better bounds**.
- **Overhead:** While the overhead in strengthening LM-Cut is negligible in our context, it might be too much for integrating this technique inside of A^* , where LM-Cut needs to be evaluated at each step and a more efficient algorithm is likely to produce better results.

Open directions:

- Optimizing the LM-Cut strengthening to reduce its overhead
- Measure the strengthening of LM-Cut inside of A^*

Selected References

- [BC11] Blai Bonet and Julio Castillo. A complete algorithm for generating landmarks. In *Proceedings of the International Conference on Automated Planning and Scheduling*. AAAI Press, 2011.
- [BG01] Blai Bonet and Héctor Geffner. Planning as heuristic search. *Artificial Intelligence*, 129, 2001.
- [BH10] Blai Bonet and Malte Helmert. Strengthening landmark heuristics via hitting sets. In Helder Coelho, Rudi Studer, and Michael J. Wooldridge, editors, *ECAI*, volume 215 of *Frontiers in Artificial Intelligence and Applications*, pages 329–334. IOS Press, 2010.
- [DM26] Salvagnin Domenico and Zanella Matteo. Mip formulations for delete-free ai planning. 2026.
- [FN71] Richard E. Fikes and Nils J. Nilsson. Strips: a new approach to the application of theorem proving to problem solving. *Artificial Intelligence*, 2:189–208, 1971.
- [FRR22] Masood Feyzbakhsh Rankooh and Jussi Rintanen. Efficient computation and informative estimation of h^+ by integer and linear programming. *Proceedings of the International Conference on Automated Planning and Scheduling*, 32, 2022.
- [HD09] Malte Helmert and Carmel Domshlak. Landmarks, critical paths and abstractions: what's the difference anyway? In *Proceedings of the International Conference on Automated Planning and Scheduling*, 2009.
- [HST12] Patrik Haslum, John Slaney, and Sylvie Thiebaux. Minimal landmarks for optimal delete-free planning. *ICAPS 2012 - Proceedings of the 22nd International Conference on Automated Planning and Scheduling*, 22, 2012.
- [LF20] Pascal Lauer and Maximilian Fickert. Beating LM-cut with LM-cut: Quick cutting and practical tie breaking for the precondition choice function. In *ICAPS 2020 Workshop on Heuristics and Search for Domain-independent Planning*, 2020.